Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4

За седьмой семестр

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: Прогнозирование временного ряда многослойным персептроном

Выполнил:
Студент 4 курса
Группы ИИ-26(2)
Семёнов А.В.
Проверил:
Никитин С.К.

Брест 2026

Прогнозирование временного ряда

1 Условие работы и вариант задания

Цель работы — реализовать многослойный персептрон p–N–1 для одношагового и рекурсивного прогнозирования временного ряда, исследовать влияние длины окна и числа скрытых нейронов на точность, скорость и устойчивость обучения. Вариант 1 сохраняется из предыдущих лабораторных работ.

$$x(k) = cos(0{,}30k) + 0{,}5\,sin(0{,}70k), \quad k = 0, \dots, 199$$

a	b	c	d	p	N
1,0	0,30	0,5	0,70	6	4

Аргументы тригонометрических функций заданы в радианах. Первые 140 значений, k = 0…139, образуют обучающий участок; следующие 60, k = 140…199, — тестовый. В базовом эксперименте используется сеть 6–4–1 со скрытой сигмоидой и линейным выходом.

Программа написана на Python. Прямой проход, обратное распространение, скользящие окна, метрики и рекурсивный прогноз реализованы самостоятельно с NumPy. Matplotlib применяется для графиков. ML-фреймворки и готовые функции построения окон не используются.

Нормализация без информации из будущего

$$u(k) = (x(k) - x_min)/(x_max - x_min)$$

$$\hat{x}(k) = x_min + (x_max - x_min)\hat{u}(k)$$

Минимум и максимум вычисляются только по x(0)…x(139): x_min = -1,475837737, x_max = 1,472943089; размах обучающего участка R_train = 2,948780827. Этими же параметрами преобразуются тестовые и пользовательские окна, а затем восстанавливается исходная шкала прогнозов.

Тестовые нормированные значения лежат в [0,010814881; 1,002738751]. Выше 1 находятся 1 точек, ниже 0 — 0. Обрезание до [0; 1] не выполняется. Линейный выход допускает прогноз за границами обучающего диапазона.

Расчёт min/max по всему ряду передал бы алгоритму информацию о будущих экстремумах. Даже без использования тестовых целей в градиенте это было бы утечкой данных через предобработку.

2 Скользящее окно и алгоритм обучения

$$X_k = [u(k-p), \ldots, u(k-1)], \quad t_k = u(k)$$

Входы перечисляются от старого значения к новому. Это обратный порядок относительно записи лагов в методичке; при согласованном порядке весов и входов задача остаётся той же. Обучающие цели: k = p…139. Тестовые цели: k = 140…199, причём первое тестовое окно использует последние p известных значений train.

Окно p	Обучающих примеров	Тестовых примеров
2	138	60
4	136	60
6	134	60
12	128	60

Для базового p = 6 первое обучающее окно x(0)…x(5) предсказывает x(6), последнее x(133)…x(138) — x(139). Первое тестовое окно x(134)…x(139) предсказывает x(140). При одношаговой проверке используются фактически наблюдённые прошлые значения, в том числе уже поступившие точки теста. В рекурсивном режиме они заменяются прогнозами.

Прямой проход и градиенты MSE

$$a_j = \Sigma_i X_i W_{ij} + b_j; \quad h_j = \sigma(a_j); \quad \sigma(a) = 1/(1 + exp(-a))$$

$$\hat{u} = \Sigma_j h_j v_j + b_out; \quad L = (\hat{u} - t)^2; \quad Es = (1/m)\Sigma_k L_k$$

Поскольку производная линейного выхода равна 1, выходная дельта совпадает с производной квадратичной потери по прогнозу. Множитель 2 сохраняется: в программе применяется MSE без коэффициента ½.

$$\delta_out = \partial L/\partial\hat{u} = 2(\hat{u} - t); \quad \delta_j = \delta_out\, v_j h_j (1 - h_j)$$

$$\partial L/\partial W_{ij} = X_i \delta_j; \quad \partial L/\partial b_j = \delta_j$$

$$\partial L/\partial v_j = h_j \delta_out; \quad \partial L/\partial b_out = \delta_out; \quad \theta \leftarrow \theta - \eta \partial L/\partial\theta$$

Все дельты вычисляются до изменения весов. Один online-шаг использует градиент одной ошибки L; среднее Es пересчитывается после эпохи при единых конечных весах и используется для остановки. Тестовые данные в обновлении весов и остановке не участвуют.

3 Параметры и проверка реализации

Параметр	Значение
Скорость обучения η	0,05
Критерий	$Es \le 10^{-4}$
Предел эпох	20 000
Seed	42, 43, 44, 45, 46
Начальные веса и смещения	U[−0,3; 0,3), default_rng(seed)
Порядок в эпохе	Случайная перестановка обучающих примеров
Генератор порядка	default_rng(seed + 100000)
Представитель конфигурации	Первый seed = 42

Выбранный шаг 0,05 находится в диапазоне 0,01–0,3 из методички. Предварительная проверка на базовой сети и seed = 42 проведена только по обучающей ошибке, с бюджетом 1000 эпох. Она показала достижение порога при 0,05 без необходимости выбирать максимальный шаг. Более быстрый шаг в одном опыте не гарантирует устойчивости для всех размеров сети; для сопоставимости во всех сериях сохранено η = 0,05.

η	Эпохи в проверке	Es на train
0,01	1000	0.000655376
0,05	310	0.000099611
0,10	176	0.000096813
0,30	100	0.000090426

Всего проведено 35 различных основных обучений: четыре размера окна при N = 4 и ещё три размера скрытого слоя при p = 6, по пять seed. Общие пять запусков 6–4–1 используются в обеих таблицах, а не повторяются. Дополнительно выполнено пять запусков базовой сети с шумом во входах.

Проверка корректности

Аналитические градиенты всех 11 параметров контрольной сети 3–2–1 проверены центральными конечными разностями с шагом 10^{-6}. Максимальное абсолютное расхождение — $1{,}551 \cdot 10^{-10}$ при допуске 10^{-8}. Проверены обратимость нормализации, границы train/test, количество окон, повторяемость обучения, пересчёт Es и остановка уже на нулевой эпохе при выполненном критерии.

Проверены обработка sMAPE для пары нулей, отклонение NaN и бесконечностей, длина пользовательского окна, а также совпадение первого рекурсивного прогноза с одношаговым. Расчёт шага по параметрам варианта отдельно воспроизведён теми же функциями, которые используются при обучении.

4 Базовый эксперимент

Результаты получены запуском lab4.py, NumPy 2.3.5. Базовая сеть 6–4–1 содержит 33 параметра. При seed = 42 обучение завершилось на эпохе 310; Es = 0,000099611. Критерий остановки достигнут.

Участок	Примеры	MAE	RMSE	sMAPE
Train	134	0,023700	0,029430	8,008%
Test	60	0,024544	0,030362	12,054%

$$MAE = (1/m)\Sigma_k |\hat{x}_k - x_k|; \quad RMSE = \sqrt{[(1/m)\Sigma_k(\hat{x}_k - x_k)^2]}$$

$$sMAPE = (100\%/m)\Sigma_k 2|\hat{x}_k - x_k|/(|x_k| + |\hat{x}_k|)$$

Все метрики вычисляются после обратного пересчёта в исходную шкалу. Ряд меняет знак и проходит через ноль, поэтому MAPE с делением на $|x_k|$ нестабильна около нуля. Используется sMAPE; если эталон и прогноз одновременно равны нулю, вклад принимается равным 0. Диапазон этой версии sMAPE — от 0 до 200%; около нуля она всё равно чувствительна, поэтому оценивается вместе с MAE и RMSE.

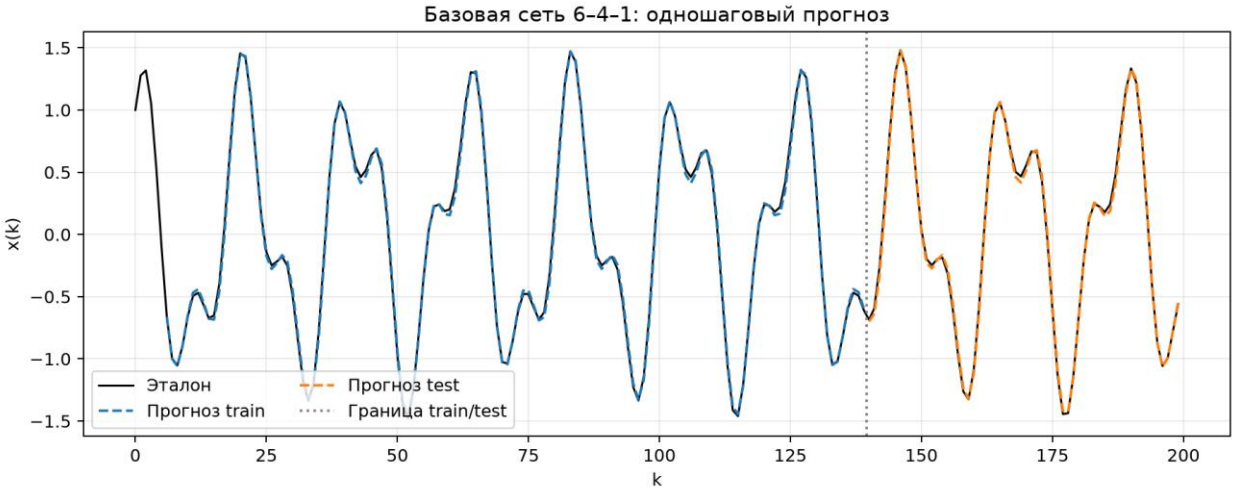


Рисунок 1 — Ряд и одношаговый прогноз базовой сети

Прогноз на обучающем участке начинается с k = 6: для первых шести точек нет полного предшествующего окна. Вертикальная линия отделяет обучающие цели от тестовых. Одношаговая оценка использует истинную историю и не равнозначна прогнозированию всех 60 точек без новых наблюдений.

5 Влияние размера окна

Число скрытых нейронов фиксировано: N = 4. Для p ∈ {2, 4, 6, 12} выполнено по пять запусков. В таблицах «± σ» обозначает стандартное отклонение по пяти значениям с ddof = 0. Время до остановки включает остановку по лимиту; 20 000 эпох нельзя трактовать как достигнутое время сходимости.

p	Эпохи, среднее ± σ	Test MAE, среднее ± σ	Не достигли Ee
2	20000,0 ± 0,0	0,094726 ± 0,003134	5/5
4	3649,0 ± 151,4	0,023823 ± 0,000102	0/5
6	305,8 ± 9,0	0,023987 ± 0,000500	0/5
12	26,6 ± 5,2	0,021609 ± 0,001318	0/5

p	Диапазон эпох	Диапазон test MAE	Средняя train MAE
2	20000–20000	0,090045–0,098923	0,095834
4	3463–3835	0,023649–0,023951	0,023279
6	293–320	0,023247–0,024544	0,023845
12	23–37	0,019616–0,023492	0,021516

В серии окон минимальная средняя тестовая MAE получена при p = 12: 0,021609. Для p = 2 ошибка равна 0,094726, для p = 6 — 0,023987, для p = 12 — 0,021609. Вывод об улучшении делается по этим измерениям, а не по предположению, что большее окно всегда лучше.

Сумма двух гармоник с различными частотами удовлетворяет линейной рекуррентности четвёртого порядка. Поэтому четыре лага уже содержат достаточную историю для точного линейного описания идеального ряда. Два лага в общем случае недостаточны для восстановления четырёхкомпонентного состояния; нелинейная сеть может приближать конечную выборку, но это не обеспечивает устойчивого продолжения ряда.

Увеличение p добавляет избыточные коррелированные признаки и входные веса, а число обучающих окон уменьшается с 138 до 128. Даже при достаточном числе лагов результат сигмоидной сети зависит от оптимизации, выбранного порога и случайных весов. Это объясняет, почему преимущество большого окна не гарантировано.

В полученных данных переход p = 4 → 6 немного увеличил среднюю MAE, а переход к p = 12 снова уменьшил её. Устойчивое плато по четырём размерам не установлено; утверждать, что после p = 4 или p = 6 улучшение окончательно прекращается, нельзя.

6 Влияние скрытого слоя и случайной инициализации

Размер окна фиксирован: p = 6. Исследованы N ∈ {1, 2, 4, 8}. Число параметров равно pN + 2N + 1: соответственно 9, 17, 33 и 65. Все пять seed, диапазон инициализации, шаг и критерий остановки одинаковы.

N	Эпохи, среднее ± σ	Test MAE, среднее ± σ	Не достигли Ee
1	350,2 ± 25,7	0,024967 ± 0,000269	0/5
2	331,4 ± 25,1	0,024533 ± 0,000132	0/5
4	305,8 ± 9,0	0,023987 ± 0,000500	0/5
8	295,6 ± 3,4	0,023923 ± 0,000519	0/5

N	Диапазон эпох	Средняя train MAE	Диапазон test MAE
1	318–382	0,024141	0,024657–0,025361
2	297–363	0,023964	0,024375–0,024735
4	293–320	0,023845	0,023247–0,024544
8	291–301	0,023789	0,023327–0,024686

Во всей основной серии 5 из 35 запусков остановились по пределу эпох. Разброс эпох и MAE показывает чувствительность обучения к инициализации и порядку примеров. Одинаковый seed не делает веса разных размерностей одинаковыми; сравниваются серии с общим правилом генерации.

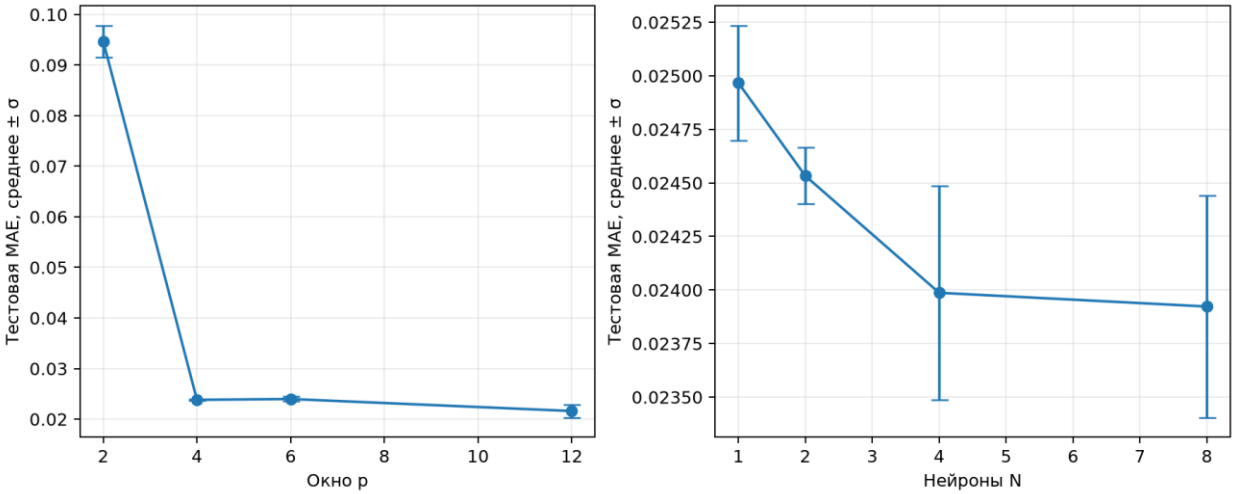


Рисунок 2 — Влияние p и N на тестовую MAE

Один скрытый нейрон задаёт сигмоиду от одной линейной комбинации лагов с последующим масштабированием. Он способен приблизить линейную рекуррентность на ограниченном диапазоне, но точности может мешать кривизна сигмоиды. Поэтому необходимость нескольких нейронов оценивается экспериментально; число гармоник само по себе не задаёт минимальное N.

В этой серии N = 1 оказалось достаточно для достижения заданного Ee во всех пяти запусках. Переход N = 4 → 8 дал лишь небольшое уменьшение средней тестовой MAE по сравнению с разбросом отдельных запусков.

7 Графики сходимости и тестового прогноза

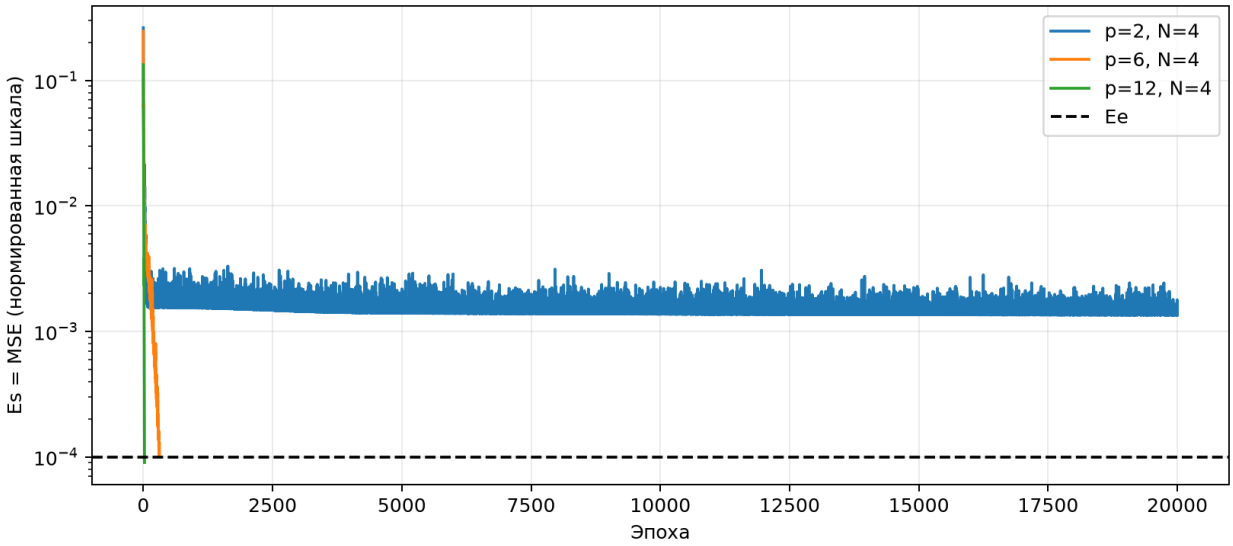


Рисунок 3 — Es для p = 2, 6 и 12 при N = 4 и seed 42

На одном графике представлены одинаковые средние квадратичные ошибки в нормированной шкале. Ось Es логарифмическая; горизонтальная линия соответствует Ee = 10^{-4}. При стохастическом порядке примеров ошибка может колебаться и не обязана убывать в каждой эпохе.

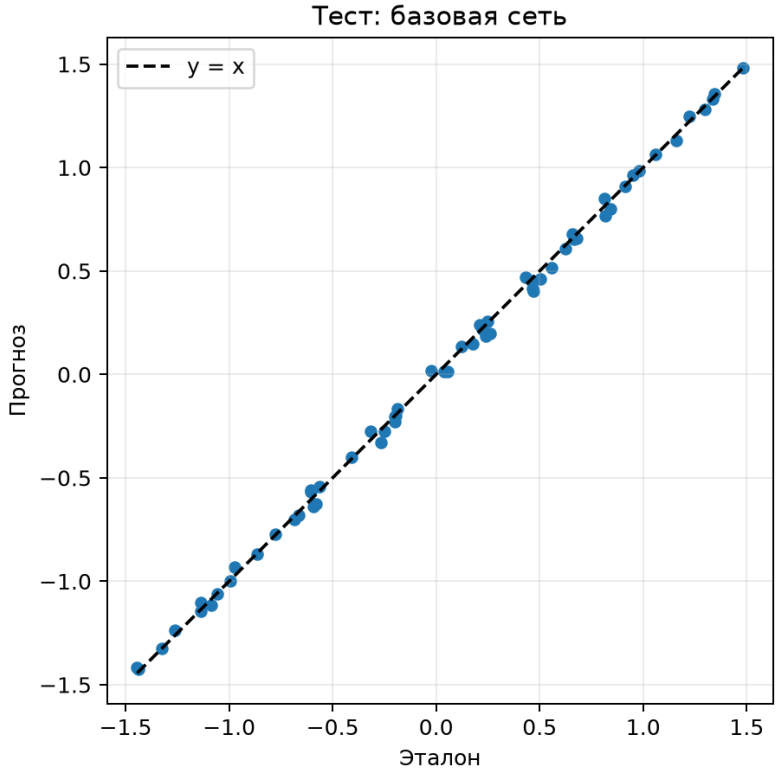


Рисунок 4 — Эталон и одношаговый прогноз на тесте

Диагональ y = x соответствует идеальному совпадению. Отклонения точек отражают ошибку базовой сети на 60 тестовых целях. Близость к диагонали сама по себе не характеризует устойчивость рекурсивной траектории.

8 Рекурсивный прогноз на 30 шагов

Лучшая конфигурация выбрана по минимальной средней тестовой MAE пяти запусков: p = 12, N = 4, средняя MAE = 0,021609. Для прогноза заранее принят первый seed = 42, без выбора наиболее удачного запуска внутри конфигурации.

Начальное окно содержит только известные значения x(128)…x(139). Шаг 1 предсказывает x(140), шаг 30 — x(169). Каждый прогноз добавляется в окно и вытесняет старейшее значение. Истинные x(140)…x(169) используются только для последующего расчёта ошибки.

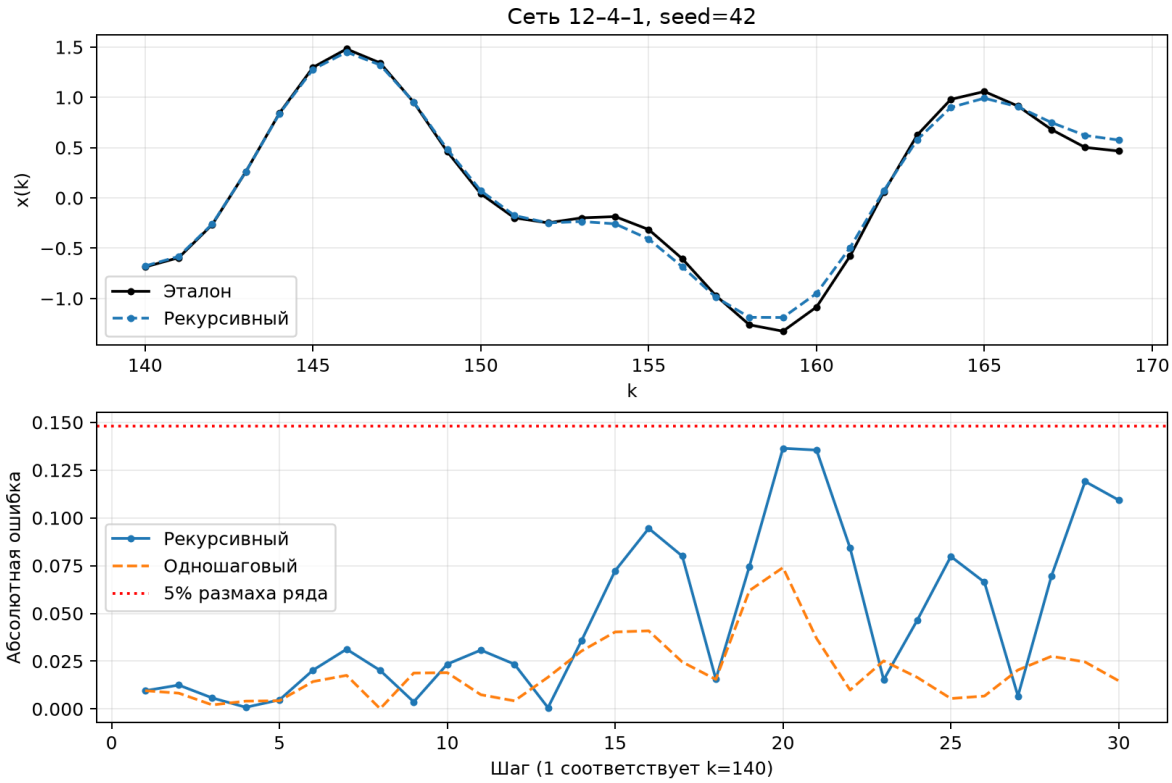


Рисунок 5 — Рекурсивная траектория и абсолютная ошибка

Порог пригодности принят равным 5% размаха всего заданного ряда из 200 точек: 0,05(max x − min x) = 0,147842840. Это диагностический порог для готового прогноза, он не участвует в обучении или нормализации.

На горизонте 30 шагов превышения порога не зарегистрировано. Это подтверждает пригодность только на проверенном горизонте, а не на любом числе будущих шагов.

На одинаковом отрезке k = 140…169 одношаговая MAE составляет 0,020054, рекурсивная — 0,047621. Ошибка обратной связи меняет будущие входы, поэтому возможны накопление фазового сдвига и изменение амплитуды. Рост не обязан быть монотонным.

9 Обобщение и эксперимент с шумом

Для дополнительной серии выбрана базовая сеть 6–4–1. К каждому нормированному обучающему входу один раз добавлен независимый гауссов шум со средним 0 и σ = 0,01. Масштаб шума в исходной шкале равен 0,01R_train. Цели, параметры нормализации и тестовые данные оставлены чистыми. Для пяти seed используются независимые генераторы seed + 200000.

Условие	Средняя train MAE	Средняя test MAE	Не достигли Ee
Без шума	0,023845	0,023987	0/5
Шум σ = 0,01	0,029434	0,030155	5/5

Seed	Эпохи с шумом	Es с шумом	Чистая test MAE
42	20000	0,000605202	0,029731
43	20000	0,000593395	0,025221
44	20000	0,000516577	0,025453
45	20000	0,000551906	0,033774
46	20000	0,000757668	0,036597

При добавлении шума средняя тестовая MAE выросла с 0,023987 до 0,030155. Это результат выбранного уровня и способа добавления шума; он не доказывает, что шум полезен или вреден при любых параметрах. Для шумных запусков Es относится к зашумлённым входам, а train/test MAE — к исходному чистому ряду.

Отношение средних test MAE к train MAE в исследованных конфигурациях находится в пределах 0,99–1,03. Эти значения позволяют оценить разрыв обобщения, но по одному разбиению нельзя строго установить отсутствие переобучения. Обе части содержат один и тот же детерминированный процесс, что облегчает перенос на тест.

Выраженного переобучения в виде существенно большей тестовой ошибки в этих сериях не наблюдается. Для p = 2 близкие ошибки train и test остаются высокими, что указывает на недостаточную точность приближения, а не на хорошее качество прогноза.

Для улучшения обобщения можно выделить временной валидационный участок внутри первых 140 точек и выбирать раннюю остановку по нему, уменьшить N, исследовать несколько уровней шума или увеличить обучающий участок при сохранении отдельного будущего теста. Слишком ранняя остановка и чрезмерное уменьшение сети могут привести к недообучению.

По требованию задания конфигурация для рекурсии выбирается по тестовой MAE. После такого выбора тест фактически играет роль валидации; его оценка для выбранной сети уже не является независимой финальной оценкой. Для строгого сравнения понадобился бы ещё один, не использованный при выборе, будущий участок.

10 Режим функционирования

В режиме функционирования используется базовая сеть 6–4–1, seed = 42. Вводятся шесть чисел в хронологическом порядке либо команда k <индекс цели>. Например, k 140 означает взять x(134)…x(139) и предсказать x(140). Такой смысл индекса явно указан в приглашении программы.

Окно	û	Прогноз x̂	Эталон x	|Ошибка|
k=20	0,9921	1,449605	1,455474	0,005869
k=70	0,1518	-1,028310	-1,024606	0,003705
k=120	0,5853	0,250031	0,238631	0,011399
k=140	0,2630	-0,700176	-0,686676	0,013500
k=165	0,8616	1,064886	1,057796	0,007090
k=199	0,3163	-0,543041	-0,561371	0,018330
Пользовательское	0,3884	-0,330390	—	—
С выбросом	-0,7417	-3,662993	—	—

k=20. X = [-0,6735; -0,6506; -0,4021; 0,0689; 0,6515; 1,1695]; u = [0,2721; 0,2798; 0,3641; 0,5239; 0,7214; 0,8971].

k=70. X = [1,3040; 1,2951; 0,9803; 0,4259; -0,2088; -0,7376]; u = [0,9427; 0,9397; 0,8329; 0,6449; 0,4297; 0,2504].

k=120. X = [-1,4128; -1,4609; -1,2023; -0,7480; -0,2682; 0,0841]; u = [0,0214; 0,0051; 0,0928; 0,2468; 0,4095; 0,5290].

k=140. X = [-1,0182; -0,8177; -0,5918; -0,4682; -0,4927; -0,6083]; u = [0,1552; 0,2232; 0,2998; 0,3417; 0,3334; 0,2942].

k=165. X = [-1,3258; -1,0851; -0,5780; 0,0545; 0,6253; 0,9797]; u = [0,0509; 0,1325; 0,3045; 0,5190; 0,7126; 0,8327].

k=199. X = [0,2119; -0,4069; -0,8651; -1,0579; -0,9929; -0,7774]; u = [0,5724; 0,3625; 0,2071; 0,1417; 0,1638; 0,2369].

Пользовательское. X = [0,1000; 0,2000; 0,4000; 0,3000; 0,1000; -0,1000]; u = [0,5344; 0,5683; 0,6361; 0,6022; 0,5344; 0,4666].

С выбросом. X = [0,1000; 0,2000; 0,4000; 4,0000; 0,1000; -0,1000]; u = [0,5344; 0,5683; 0,6361; 1,8570; 0,5344; 0,4666].

Первые три примера относятся к train, следующие три — к test. Для двух пользовательских окон эталон не задан, поэтому ошибка не вычисляется. Выброс 4,0 находится вне обучающего диапазона; нормализация не обрезает его, и ответ показывает экстраполяцию сети, а не гарантированно верный прогноз.

11 Блок-схема обработки данных

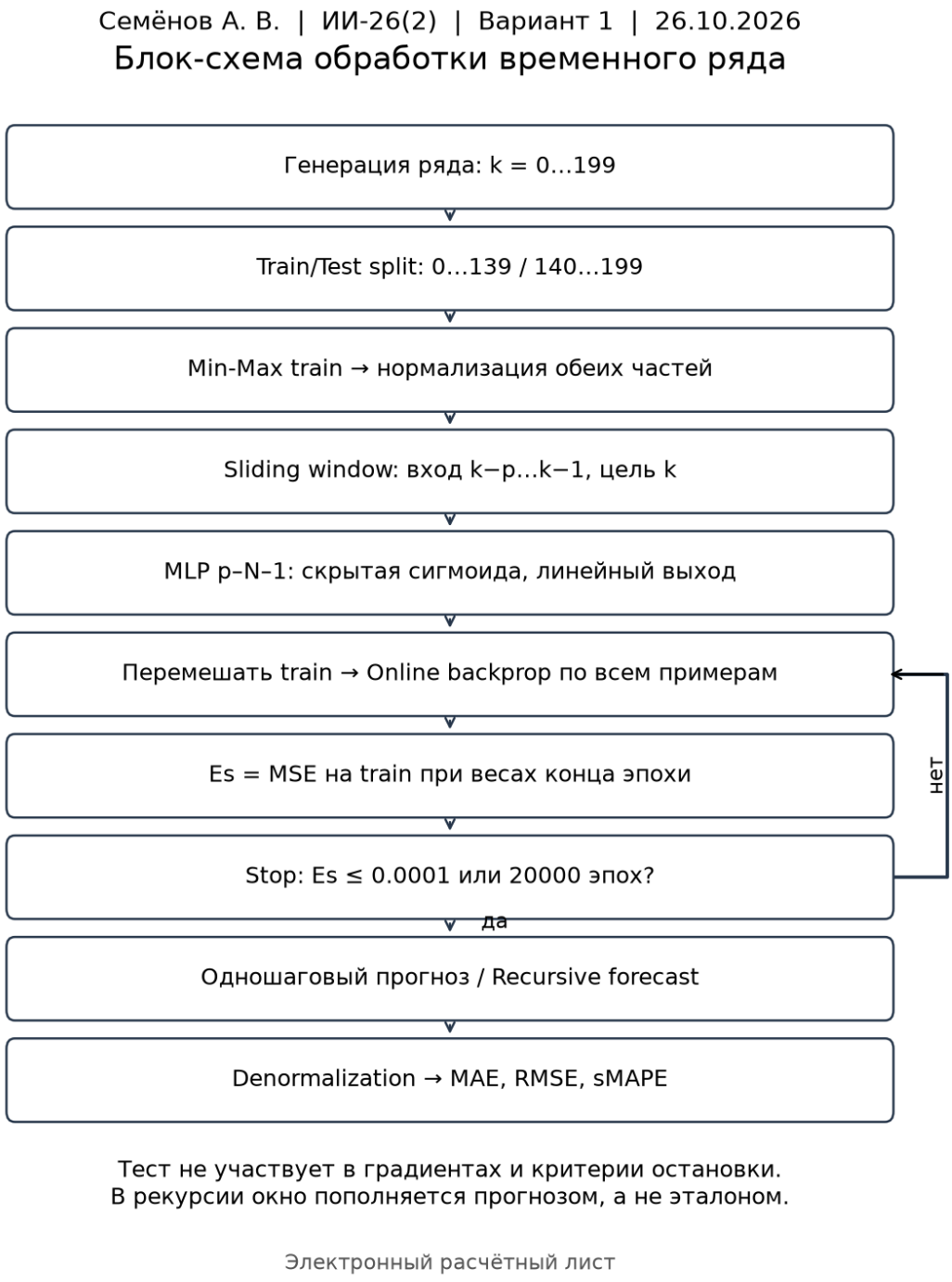


Рисунок 6 — Электронный лист блок-схемы

Сначала выделяется train, затем по нему рассчитывается Min-Max. Sliding window предшествует обучению MLP, обновления выполняются online, после эпохи проверяется средняя Es. При продолжении обучения порядок примеров снова перемешивается. После прогноза выполняются Denormalization и расчёт MAE, RMSE, sMAPE.

Блоки Normalize using whole dataset, Batch update и Sigmoid output исключены: они противоречат разбиению без утечки, online-обучению и линейному выходу. MAPE заменён на sMAPE из-за переходов ряда через ноль.

12 Расчёт одного шага обучения

Семёнов А. В. | ИИ-26(2) | Вариант 1 | 26.10.2026

Один шаг обучения сети 3–2–1

$X = [0.2; 0.5; 0.8]$, $d = 0.6$, $V = 1$

$\eta = 0.05$; $L = (\hat{y} - d)^2$; $\theta_{new} = \theta - \eta \cdot \partial L/\partial\theta$

$W = [[0.05, -0.2], [0.3, 0.05], [-0.1, 0.4]]$

$b = [0.1, -0.1]$; $v = [0.5, -0.2]$; $b_{out} = 0.2$

1. Прямой проход

$a_1 = 0.2 \cdot 0.05 + 0.5 \cdot 0.3 + 0.8 \cdot (-0.1) + 0.1 = 0.18$

$a_2 = 0.2 \cdot (-0.2) + 0.5 \cdot 0.05 + 0.8 \cdot 0.4 - 0.1 = 0.205$

$h_1 = \sigma(a_1) = 0.544878892$; $h_2 = \sigma(a_2) = 0.551071269$

$\hat{y} = 0.5h_1 - 0.2h_2 + 0.2 = 0.362225192$

$L = (\hat{y} - 0.6)^2 = 0.056536859$

2. Обратный проход со старыми выходными весами

$\delta_{out} = 2(\hat{y} - d) = -0.475549615$

$\delta_1 = \delta_{out} \cdot 0.5 \cdot h_1(1-h_1) = -0.058964796$

$\delta_2 = \delta_{out} \cdot (-0.2) \cdot h_2(1-h_2) = 0.023529408$

$\partial L/\partial v = [-0.259116948; -0.262061730]$, $\partial L/\partial b_{out} = \delta_{out}$

$\partial L/\partial W_{ij} = X_i\delta_j$; $\partial L/\partial b_j = \delta_j$

$\partial L/\partial W$, строка 1: [-0.011792959; 0.004705882]

$\partial L/\partial W$, строка 2: [-0.029482398; 0.011764704]

$\partial L/\partial W$, строка 3: [-0.047171837; 0.018823526]

3. Обновлённые параметры

W_{new}, строка 1: [0.050589648; -0.200235294]

W_{new}, строка 2: [0.301474120; 0.049411765]

W_{new}, строка 3: [-0.097641408; 0.399058824]

$b_{new} = [0.102948240; -0.101176470]$

$v_{new} = [0.512955847; -0.186896913]$, $b_{out_new} = 0.223777481$

Проверка: $\hat{y}_{new} = 0.401111400$

$L_{new} = 0.039556675 < 0.056536859$

Электронный расчётный лист

Рисунок 7 — Электронный расчётный лист для варианта 1

В контрольной сети 3–2–1 после полного прямого и обратного прохода получено $\hat{u} = 0{,}362225192$, $L = 0{,}056536859$. После одного online-обновления при $\eta = 0{,}05$: $\hat{u}_{new} = 0{,}401111400$, $L_{new} = 0{,}039556675$. Все промежуточные вычисления и новые веса приведены на листе.

13 Выводы

Размер окна. Минимальная средняя test MAE в серии окон получена при p = 12. Значения для p = 2, 4, 6, 12 составили 0,094726, 0,023823, 0,023987, 0,021609. Двух лагов недостаточно для общего линейного описания двух гармоник; четыре лага теоретически достаточны, но обучаемая сигмоидная сеть лишь приближает эту зависимость. Большое окно добавляет избыточность и не гарантирует улучшения.

Размер скрытого слоя. В серии N наименьшая средняя test MAE получена при N = 8. При N = 1 она равна 0,024967, порог не достигнут в 0/5 запусков. Скорость и точность зависят от числа параметров и оптимизации; нельзя заранее считать большее N лучшим.

Линейный выход. Линейная активация допускает любой действительный прогноз, включая выход за диапазон train. Сигмоида ограничила бы нормированный ответ интервалом (0; 1), а восстановленный — обучающими экстремумами. При ненормированных целях отрицательные значения данного ряда вообще недостижимы для сигмоидного выхода. Линейная сеть может обучаться на исходной шкале, но масштаб градиентов и подходящий шаг изменятся.

Нормализация. Min-Max по train исключает утечку будущих экстремумов. На тесте максимум u = 1,002739 > 1, поэтому ограничивать нормированные данные нельзя. Выход за диапазон означает экстраполяцию, а не ошибку преобразования.

Инициализация. В основных сериях 5/35 запусков не достигли Ee за 20 000 эпох. Разброс результатов отражён в таблицах. Малый шаг замедляет обучение, большой может усилить колебания; p и N меняют выразительность и геометрию задачи. Пять seed характеризуют эти серии, а не все возможные инициализации.

Рекурсивный горизонт. За 30 шагов порог 5% размаха не превышен. Рекурсивная MAE на этом отрезке 0,047621, одношаговая 0,020054. Собственные ошибки сети возвращаются во вход и изменяют последующие прогнозы.

Обобщение и шум. При шуме σ = 0,01 тестовая MAE базовой сети выросла: 0,023987 → 0,030155. Регуляризацию следует проверять по валидации. Ранняя остановка, изменение N и увеличение выборки могут помочь, но их эффект зависит от режима обучения; подбор по текущему тесту требует отдельной финальной проверки.

Связь с предыдущими лабораторными работами

В ЛР №1 XOR показал необходимость нелинейного скрытого слоя: один пороговый сигмоидный нейрон задаёт прямую границу и не разделяет противоположные вершины. Обратимое кодирование классов 0 и −6 не устраняет линейную неразделимость. В текущей работе скрытый слой также формирует признаки, но задача стала регрессионной.

В ЛР №2 сравнивались квадратичная ошибка и BCE для бинарной цели. Для сигмоидного выхода BCE даёт $\delta = p - t$, а квадратичная ошибка сохраняет множитель $p(1 - p)$; это влияет на скорость, но не гарантирует глобальную сходимость. В текущей работе цель непрерывна, выбран линейный выход и MSE; выходная производная равна 1.

В ЛР №3 ReLU имела производную 1 на положительной ветви и 0 на отрицательной. Это уменьшает затухание градиента на активной ветви, но может создавать нейроны, неактивные на всей выборке. У Сигмоиды производная $h(1 - h) \leq ¼$ и уменьшается в насыщении. В текущей работе скрытая Сигмоида задана методичкой; малые начальные веса и нормализация помогают не начинать обучение в сильном насыщении.

Во всех работах сохраняются прямой проход, вычисление дельт со старыми весами, обновление параметров и независимый пересчёт критерия после эпохи. Accuracy относится к классификации XOR; для непрерывного ряда используются MAE, RMSE и sMAPE. График одношагового прогноза, как и карта классов, дополняет численные метрики, но не заменяет проверку на новых данных.

Запуск и воспроизведение

```
python -m pip install numpy matplotlib
python lab4.py
python lab4.py --self-test
python lab4.py --no-show --no-input
```

Файл lab4.py автономен и не импортирует код предыдущих работ. При обычном запуске выполняется вся серия, сохраняются результаты и открываются графики; после закрытия окон доступен ввод. Полная серия может занять несколько минут из-за запусков до предела 20 000 эпох. Параметры --lr, --ee, --max-epochs, --seeds и --out доступны для дополнительных опытов; числа отчёта относятся к значениям по умолчанию.

В папке lab4_results сохраняются results.json с весами и историями, две CSV-таблицы, пять исследовательских графиков и два электронных расчётных листа. Команда q завершает ввод; десятичная запятая поддерживается. Неверная длина окна, NaN и бесконечность отклоняются.

Приложение А Результаты отдельных запусков

Исследование окна

p	N	Seed	Эпохи	Es	Train MAE	Test MAE
2	4	42	20000	0,001349903	0,091095	0,090045
2	4	43	20000	0,001637583	0,099746	0,096638
2	4	44	20000	0,001625073	0,099144	0,095581
2	4	45	20000	0,001586559	0,098008	0,098923
2	4	46	20000	0,001401355	0,091176	0,092443
4	4	42	3835	0,000099891	0,023391	0,023894
4	4	43	3604	0,000099024	0,023347	0,023827
4	4	44	3463	0,000099874	0,023138	0,023649
4	4	45	3526	0,000099258	0,023312	0,023951
4	4	46	3817	0,000099294	0,023205	0,023796
6	4	42	310	0,000099611	0,023700	0,024544
6	4	43	320	0,000098725	0,023369	0,024246
6	4	44	305	0,000098853	0,024085	0,023247
6	4	45	301	0,000098623	0,024170	0,023547
6	4	46	293	0,000099437	0,023903	0,024352
12	4	42	23	0,000090305	0,020645	0,020833
12	4	43	24	0,000097244	0,023779	0,023492
12	4	44	37	0,000092910	0,021468	0,021764
12	4	45	24	0,000081804	0,022147	0,022340
12	4	46	25	0,000094558	0,019542	0,019616

Es > 0,0001 при 20 000 эпох означает остановку по лимиту. Пять результатов 6–4–1 из таблицы окна входят также в исследование скрытого слоя.

Дополнительные размеры скрытого слоя

p	N	Seed	Эпохи	Es	Train MAE	Test MAE
6	1	42	318	0,000099175	0,024292	0,025076
6	1	43	382	0,000099662	0,024137	0,025361
6	1	44	340	0,000097773	0,024193	0,024671
6	1	45	332	0,000098258	0,024064	0,024657
6	1	46	379	0,000099986	0,024021	0,025069
6	2	42	297	0,000098668	0,023867	0,024416
6	2	43	345	0,000096326	0,023909	0,024523
6	2	44	307	0,000097225	0,023871	0,024375
6	2	45	345	0,000096174	0,023857	0,024735
6	2	46	363	0,000096857	0,024316	0,024616
6	8	42	301	0,000091357	0,023213	0,023353
6	8	43	291	0,000097913	0,024010	0,023327
6	8	44	296	0,000097821	0,023906	0,024076
6	8	45	297	0,000099318	0,024204	0,024686
6	8	46	293	0,000098257	0,023613	0,024171

Es > 0,0001 при 20 000 эпох означает остановку по лимиту. Пять результатов 6–4–1 из таблицы окна входят также в исследование скрытого слоя.

Приложение Б Полный код программы

```
"""ЛР 4, вариант 1. Прогнозирование суммы двух гармоник сетью p-N-1.

Установка: python -m pip install numpy matplotlib
Запуск: python lab4.py
Без окон и ввода: python lab4.py --no-show --no-input
Проверки: python lab4.py --self-test
Все алгоритмы находятся в этом файле. ML-фреймворки не используются.
"""
import argparse
import csv
import json
from pathlib import Path

import numpy as np

SPLIT, LENGTH = 140, 200
P_BASE, N_BASE = 6, 4
SEEDS = [42, 43, 44, 45, 46]

def generate_series():
    k = np.arange(LENGTH, dtype=float)
    return np.cos(.30 * k) + .5 * np.sin(.70 * k)

def fit_scaler(train):
    low, high = float(np.min(train)), float(np.max(train))
    if not np.isfinite([low, high]).all() or high <= low:
        raise ValueError('Для нормализации нужен конечный непостоянный ряд.')
    return low, high

def normalize(values, scaler):
    low, high = scaler
    return (np.asarray(values, dtype=float) - low) / (high - low)

def denormalize(values, scaler):
    low, high = scaler
    return np.asarray(values) * (high - low) + low

def make_windows(series, p, start, stop):
    """Цели k=start..stop-1; окно в хронологическом порядке k-p..k-1."""
    if not (1 <= p <= start < stop <= len(series)):
        raise ValueError('Некорректные границы окон.')
    x = np.empty((stop-start, p))
    y = np.empty(stop-start)
    for row, k in enumerate(range(start, stop)):
        x[row] = series[k-p:k]
        y[row] = series[k]
    return x, y

def sigmoid(z):
    e = np.exp(-np.abs(z))
    return np.where(z >= 0., 1. / (1. + e), e / (1. + e))

class MLP:
    def __init__(self, p, n, seed):
        rng = np.random.default_rng(seed)
        self.w1 = rng.uniform(-.3, .3, (p, n))
        self.b1 = rng.uniform(-.3, .3, n)
        self.w2 = rng.uniform(-.3, .3, n)
        self.b2 = np.array(rng.uniform(-.3, .3))

    def parameters(self):
        return self.w1, self.b1, self.w2, self.b2

    def forward(self, x):
        h = sigmoid(np.asarray(x) @ self.w1 + self.b1)
        return h, h @ self.w2 + self.b2  # Линейный выход.

    def gradients(self, x, target):
        h, prediction = self.forward(x)
        delta = 2. * (prediction - target)  # L=(prediction-target)^2
        hidden_delta = delta * self.w2 * h * (1. - h)
        return (x[:, None] * hidden_delta, hidden_delta,
                delta * h, np.asarray(delta))

    def step(self, x, target, lr):
```

```
        # Скрытая дельта вычислена до обновления выходных весов.
        gradients = self.gradients(x, target)
        for param, grad in zip(self.parameters(), gradients):
            param -= lr * grad

    def to_list(self):
        return [param.tolist() for param in self.parameters()]

def mse(model, x, y):
    return float(np.mean((model.forward(x)[1] - y) ** 2))

def metrics(truth, prediction):
    truth, prediction = np.asarray(truth), np.asarray(prediction)
    error = np.abs(prediction-truth)
    denom = np.abs(truth) + np.abs(prediction)
    ratios = np.divide(2.*error, denom, out=np.zeros_like(error), where=denom > 0)
    return {'MAE': float(np.mean(error)),
            'RMSE': float(np.sqrt(np.mean(error**2))),
            'sMAPE': float(100.*np.mean(ratios))}

def train(x, y, n, seed, lr, ee, max_epochs):
    model = MLP(x.shape[1], n, seed)
    # Отдельный генератор: порядок эпох одинаков для одинаковых p и seed.
    order_rng = np.random.default_rng(seed + 100000)
    history = [mse(model, x, y)]
    while history[-1] > ee and len(history)-1 < max_epochs:
        for i in order_rng.permutation(len(y)):
            model.step(x[i], y[i], lr)
        error = mse(model, x, y)
        if not np.isfinite(error):
            raise FloatingPointError('Обучение разошлось; уменьшите --lr.')
        history.append(error)
    return model, history

def run_experiment(series, scaler, p, n, seed, args, noise=0.):
    z = normalize(series, scaler)
    x_train, y_train = make_windows(z, p, p, SPLIT)
    x_test, y_test = make_windows(z, p, SPLIT, LENGTH)
    fitting_x = x_train.copy()
    if noise:
        # Только обучающие входы. Цели и тестовые данные не изменяются.
        rng = np.random.default_rng(seed + 200000)
        fitting_x += rng.normal(0., noise, fitting_x.shape)
    model, history = train(fitting_x, y_train, n, seed,
                           args.lr, args.ee, args.max_epochs)
    train_pred = denormalize(model.forward(x_train)[1], scaler)
    test_pred = denormalize(model.forward(x_test)[1], scaler)
    return {'p': p, 'n': n, 'seed': seed, 'noise': noise,
            'epochs': len(history)-1, 'Es': history[-1],
            'converged': history[-1] <= args.ee, 'history': history,
            'train_count': len(y_train), 'test_count': len(y_test),
            'train': metrics(series[p:SPLIT], train_pred),
            'test': metrics(series[SPLIT:], test_pred),
            'train_prediction': train_pred.tolist(),
            'test_prediction': test_pred.tolist(), 'weights': model.to_list()}

def restore(result):
    model = MLP(result['p'], result['n'], result['seed'])
    for param, value in zip(model.parameters(), result['weights']):
        param[...] = value
    return model

def recursive_forecast(model, initial_window, scaler, steps=30):
    window = normalize(initial_window, scaler).copy()
    result = []
    for _ in range(steps):
        prediction = float(model.forward(window)[1])
        result.append(float(denormalize(prediction, scaler)))
        window[:-1] = window[1:]
        window[-1] = prediction  # Только прогноз, без подстановки эталона.
    return np.array(result)

def predict_window(model, values, scaler):
    values = np.asarray(values, dtype=float)
```

```
    if values.shape != (model.w1.shape[0],) or not np.isfinite(values).all():
        raise ValueError(f'Нужно {model.w1.shape[0]} конечных чисел.')
    normalized = normalize(values, scaler)
    with np.errstate(over='raise', invalid='raise'):
        prediction = float(model.forward(normalized)[1])
    return {'window': values.tolist(), 'normalized': normalized.tolist(),
            'prediction_normalized': prediction,
            'prediction': float(denormalize(prediction, scaler))}

def demonstrations(model, series, scaler):
    rows = []
    for k in [20, 70, 120, 140, 165, 199]:
        row = predict_window(model, series[k-P_BASE:k], scaler)
        row.update(label=f'k={k}', k=k, truth=float(series[k]),
                   absolute_error=abs(row['prediction']-float(series[k])))
        rows.append(row)
    for label, window in [('Пользовательское', [.1, .2, .4, .3, .1, -.1]),
                          ('С выбросом', [.1, .2, .4, 4., .1, -.1])]:
        row = predict_window(model, window, scaler)
        row.update(label=label, k=None, truth=None, absolute_error=None)
        rows.append(row)
    return rows

def manual_step(lr=.05):
    """Параметры из задания для V=1; тот же градиент MSE, что в обучении."""
    model = MLP(3, 2, 0)
    model.w1[:] = [[.05, -.2], [.3, .05], [-.1, .4]]
    model.b1[:] = [.1, -.1]
    model.w2[:] = [.5, -.2]; model.b2[...] = .2
    x, target = np.array([.2, .5, .8]), .6
    a = x @ model.w1 + model.b1
    h, pred = model.forward(x)
    delta = 2.*(pred-target)
    hidden_delta = delta*model.w2*h*(1-h)
    gradients = model.gradients(x, target)
    before = model.to_list()
    model.step(x, target, lr)
    return {'lr': lr, 'a': a.tolist(), 'h': h.tolist(), 'prediction': float(pred),
            'loss': float((pred-target)**2), 'delta': float(delta),
            'hidden_delta': hidden_delta.tolist(), 'before': before,
            'gradients': [v.tolist() for v in gradients], 'after': model.to_list(),
            'prediction_after': float(model.forward(x)[1])}

def summary(rows):
    epochs = np.array([r['epochs'] for r in rows])
    maes = np.array([r['test']['MAE'] for r in rows])
    return {'p': rows[0]['p'], 'n': rows[0]['n'], 'runs': len(rows),
            'epochs_mean': float(epochs.mean()), 'epochs_std': float(epochs.std()),
            'epochs_min': int(epochs.min()), 'epochs_max': int(epochs.max()),
            'test_mae_mean': float(maes.mean()), 'test_mae_std': float(maes.std()),
            'test_mae_min': float(maes.min()), 'test_mae_max': float(maes.max()),
            'train_mae_mean': float(np.mean([r['train']['MAE'] for r in rows])),
            'failures': sum(not r['converged'] for r in rows)}

def visualize(data, out, show):
    import matplotlib
    if not show:
        matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    plt.rcParams.update({'font.family': 'DejaVu Sans', 'font.size': 10})
    series = np.array(data['series']); base = data['base']
    def save(fig, filename):
        fig.savefig(out/filename, dpi=180, bbox_inches='tight')
    fig, ax = plt.subplots(figsize=(10, 4), layout='constrained')
    ax.plot(series, color='black', lw=1.2, label='Эталон')
    ax.plot(range(P_BASE, SPLIT), base['train_prediction'], '--', label='Прогноз train')
    ax.plot(range(SPLIT, LENGTH), base['test_prediction'], '--', label='Прогноз test')
    ax.axvline(139.5, color='gray', ls=':', label='Граница train/test')
    ax.set(xlabel='k', ylabel='x(k)', title='Базовая сеть 6–4–1: одношаговый прогноз')
    ax.legend(ncol=2); ax.grid(alpha=.25); save(fig, 'series.png')
    fig, ax = plt.subplots(figsize=(9, 4), layout='constrained')
    for p in [2, P_BASE, 2*P_BASE]:
        row = next(r for r in data['runs'] if r['p']==p and r['n']==N_BASE
```

```
                and r['seed']==data['parameters']['seeds'][0])
        ax.semilogy(row['history'], label=f'p={p}, N=4')
    ax.axhline(data['parameters']['ee'], color='black', ls='--', label='Ee')
    ax.set(xlabel='Эпоха', ylabel='Es = MSE (нормированная шкала)')
    ax.legend(); ax.grid(alpha=.25); save(fig, 'convergence.png')
    fig, ax = plt.subplots(figsize=(5, 5), layout='constrained')
    ax.scatter(series[SPLIT:], base['test_prediction'], s=23)
    low = min(series[SPLIT:].min(), min(base['test_prediction']))
    high = max(series[SPLIT:].max(), max(base['test_prediction']))
    ax.plot([low, high], [low, high], 'k--', label='y = x')
    ax.set(xlabel='Эталон', ylabel='Прогноз', title='Тест: базовая сеть', aspect='equal')
    ax.legend(); ax.grid(alpha=.25); save(fig, 'scatter.png')
    rec = data['recursive']; steps = np.arange(1, 31)
    fig, axes = plt.subplots(2, 1, figsize=(9, 6), layout='constrained')
    axes[0].plot(range(140, 170), series[140:170], 'k-o', ms=3, label='Эталон')
    axes[0].plot(range(140, 170), rec['prediction'], '--o', ms=3, label='Рекурсивный')
    axes[0].set(xlabel='k', ylabel='x(k)',
                title=f'Сеть {rec["p"]}–{rec["n"]}–1, seed={rec["seed"]}')
    axes[1].plot(steps, rec['absolute_error'], '-o', ms=3, label='Рекурсивный')
    axes[1].plot(steps, rec['one_step_error'], '--', label='Одношаговый')
    axes[1].axhline(rec['threshold'], color='red', ls=':', label='5% размаха ряда')
    axes[1].set(xlabel='Шаг (1 соответствует k=140)', ylabel='Абсолютная ошибка')
    for ax in axes:
        ax.legend(); ax.grid(alpha=.25)
    save(fig, 'recursive.png')
    fig, axes = plt.subplots(1, 2, figsize=(10, 4), layout='constrained')
    for ax, field, key, xlabel in [(axes[0], 'window_summary', 'p', 'Окно p'),
                                   (axes[1], 'hidden_summary', 'n', 'Нейроны N')]:
        rows = data[field]
        ax.errorbar([r[key] for r in rows], [r['test_mae_mean'] for r in rows],
                    yerr=[r['test_mae_std'] for r in rows], fmt='o-', capsize=5)
        ax.set(xlabel=xlabel, ylabel='Тестовая MAE, среднее ± σ')
        ax.grid(alpha=.25)
    save(fig, 'studies.png')
    calculation_sheets(data, out)
    if show:
        plt.show()
    plt.close('all')

def calculation_sheets(data, out):
    """Электронные расчётные листы; не имитация фотографий рукописи."""
    import matplotlib.pyplot as plt
    from matplotlib.patches import FancyBboxPatch
    settings = data.get('parameters', {'ee': .0001, 'max_epochs': 20000})
    header = 'Семёнов А. В.  |  ИИ-26(2)  |  Вариант 1  |  26.10.2026'
    fig, ax = plt.subplots(figsize=(8.2, 10.4))
    ax.set(xlim=(0, 1), ylim=(0, 1)); ax.axis('off')
    ax.text(.5, .98, header, ha='center', va='top', fontsize=11)
    ax.text(.5, .935, 'Блок-схема обработки временного ряда', ha='center', fontsize=14)
    labels = ['Генерация ряда: k = 0…199', 'Train/Test split: 0…139 / 140…199',
              'Min-Max train → нормализация обеих частей',
              'Sliding window: вход k–p…k–1, цель k',
              'MLP p–N–1: скрытая сигмоида, линейный выход',
              'Перемешать train → Online backprop по всем примерам',
              'Es = MSE на train при весах конца эпохи',
              f'Stop: Es ≤ {settings["ee"]:g} или {settings["max_epochs"]} эпох?',
              'Одношаговый прогноз / Recursive forecast',
              'Denormalization → MAE, RMSE, sMAPE']
    ys = np.linspace(.86, .16, len(labels))
    for i,(label,y) in enumerate(zip(labels,ys)):
        ax.add_patch(FancyBboxPatch((.08,y-.024), .84,.048,
                     boxstyle='round,pad=0.008',facecolor='white',edgecolor='#26364a'))
        ax.text(.5,y,label,ha='center',va='center',fontsize=10)
        if i:
            ax.annotate('',xy=(.5,y+.032),xytext=(.5,ys[i-1]-.032),
                        arrowprops={'arrowstyle':'->','color':'#26364a'})
    ax.plot([.93,.98,.98,.93],[ys[7],ys[7],ys[5],ys[5]],color='#26364a')
    ax.annotate('',xy=(.92,ys[5]),xytext=(.98,ys[5]),arrowprops={'arrowstyle':'->'})
    ax.text(.98,(ys[5]+ys[7])/2,'нет',rotation=90,ha='right',va='center',fontsize=9)
    ax.text(.53,(ys[7]+ys[8])/2,'да',fontsize=9)
    ax.text(.5,.065,'Тест не участвует в градиентах и критерии остановки.\n'
            'В рекурсии окно пополняется прогнозом, а не эталоном.',ha='center',fontsize=10)
    ax.text(.5,.015,'Электронный расчётный лист',ha='center',fontsize=9,color='#555555')
```

```
    fig.savefig(out/'pipeline.png',dpi=180,bbox_inches='tight')
    m=data['manual']; w,b,v,bo=m['after']; gw,gb,gv,gbo=m['gradients']
    lines = [
        'X = [0.2; 0.5; 0.8],  d = 0.6,  V = 1',
        f'η = {m["lr"]:g};  L = (ŷ − d)²;  θ_new = θ − η · ∂L/∂θ',
        'W = [[0.05, −0.2], [0.3, 0.05], [−0.1, 0.4]]',
        'b = [0.1, −0.1];  v = [0.5, −0.2];  b_out = 0.2',
        '', '1. Прямой проход',
        'a₁ = 0.2·0.05 + 0.5·0.3 + 0.8·(−0.1) + 0.1 = 0.18',
        'a₂ = 0.2·(−0.2) + 0.5·0.05 + 0.8·0.4 − 0.1 = 0.205',
        f'h₁ = σ(a₁) = {m["h"][0]:.9f};  h₂ = σ(a₂) = {m["h"][1]:.9f}',
        f'ŷ = 0.5h₁ − 0.2h₂ + 0.2 = {m["prediction"]:.9f}',
        f'L = (ŷ − 0.6)² = {m["loss"]:.9f}',
        '', '2. Обратный проход со старыми выходными весами',
        f'δ_out = 2(ŷ − d) = {m["delta"]:.9f}',
        f'δ₁ = δ_out · 0.5 · h₁(1−h₁) = {m["hidden_delta"][0]:.9f}',
        f'δ₂ = δ_out · (−0.2) · h₂(1−h₂) = {m["hidden_delta"][1]:.9f}',
        f'∂L/∂v = [{gv[0]:.9f}; {gv[1]:.9f}],  ∂L/∂b_out = δ_out',
        '∂L/∂Wᵢⱼ = Xᵢδⱼ;  ∂L/∂bⱼ = δⱼ',
        *[f'∂L/∂W, строка {i+1}: [{row[0]:.9f}; {row[1]:.9f}]'
          for i,row in enumerate(gw)],
        '', '3. Обновлённые параметры',
        *[f'W_new, строка {i+1}: [{row[0]:.9f}; {row[1]:.9f}]'
          for i,row in enumerate(w)],
        f'b_new = [{b[0]:.9f}; {b[1]:.9f}]',
        f'v_new = [{v[0]:.9f}; {v[1]:.9f}],  b_out_new = {bo:.9f}',
        f'Проверка: ŷ_new = {m["prediction_after"]:.9f}',
        f'L_new = {(m["prediction_after"]-.6)**2:.9f} < {m["loss"]:.9f}']
    fig, ax=plt.subplots(figsize=(8.2,10.4)); ax.axis('off')
    ax.text(.5,1.,header,ha='center',va='top',fontsize=11)
    ax.text(.5,.96,'Один шаг обучения сети 3–2–1',ha='center',fontsize=14)
    for y,line in zip(np.linspace(.915,.06,len(lines)),lines):
        ax.text(.015,y,line,fontsize=10,va='center',fontfamily='DejaVu Sans')
    ax.text(.5,.01,'Электронный расчётный лист',ha='center',fontsize=9,color='#555555')
    fig.savefig(out/'manual_step.png',dpi=180,bbox_inches='tight')

def interactive(model, series, scaler):
    p = model.w1.shape[0]
    print(f'\nБазовая сеть: введите {p} чисел от старого к новому.')
    print('Или k 140: прогноз x(140) по значениям x(134)..x(139). q — выход.')
    while True:
        try:
            line = input('> ').strip()
        except (EOFError, KeyboardInterrupt):
            print(); break
        if line.lower() in {'q', 'exit', 'выход'}:
            break
        try:
            parts = line.replace(',', '.').split()
            k = None
            if parts and parts[0].lower() == 'k':
                if len(parts) != 2:
                    raise ValueError('Формат: k 140')
                k = int(parts[1])
                if not p <= k < LENGTH:
                    raise ValueError(f'Индекс цели k должен быть от {p} до 199.')
                values = series[k-p:k]
            else:
                values = list(map(float, parts))
            row = predict_window(model, values, scaler)
            print('Нормированные входы:', np.round(row['normalized'], 6))
            print(f'Прогноз норм.: {row["prediction_normalized"]:.4f}; '
                  f'исходный: {row["prediction"]:.8f}')
            if k is not None:
                print(f'Эталон x({k})={series[k]:.8f}; '
                      f'ошибка={abs(row["prediction"]-series[k]):.8f}')
        except (ValueError, FloatingPointError, OverflowError) as exc:
            print(f'Ошибка ввода или вычисления: {exc}')

def self_test():
    series = generate_series(); scaler = fit_scaler(series[:SPLIT])
    z = normalize(series, scaler)
    assert np.allclose(denormalize(z, scaler), series)
```

```
    for p in [2, 4, 6, 12]:
        x, y = make_windows(z, p, p, SPLIT)
        tx, ty = make_windows(z, p, SPLIT, LENGTH)
        assert len(y)==140-p and len(ty)==60
        assert np.array_equal(tx[0], z[140-p:140]) and ty[0]==z[140]
    model = MLP(3, 2, 42); x = np.array([.2, .5, .8]); t = .6
    grads = model.gradients(x, t); max_error = 0.
    for param, grad in zip(model.parameters(), grads):
        for idx in np.ndindex(param.shape):
            old = float(param[idx]); eps = 1e-6
            param[idx] = old+eps; plus = float((model.forward(x)[1]-t)**2)
            param[idx] = old-eps; minus = float((model.forward(x)[1]-t)**2)
            param[idx] = old
            max_error = max(max_error, abs((plus-minus)/(2*eps)-grad[idx]))
    assert max_error < 1e-8
    assert metrics(np.array([0., 1.]), np.array([0., 1.])) == dict(MAE=0., RMSE=0., sMAPE=0.)
    assert metrics(np.array([0.]), np.array([1.]))['sMAPE']==200.
    x, y = make_windows(z, 6, 6, 140)
    a, ha = train(x, y, 4, 42, .05, 1e-8, 3)
    b, hb = train(x, y, 4, 42, .05, 1e-8, 3)
    assert ha==hb and a.to_list()==b.to_list() and len(ha)==4
    assert ha[-1]==mse(a,x,y)
    _, early = train(x, y, 4, 42, .05, 100., 3)
    assert len(early)==1
    first = recursive_forecast(a, series[134:140], scaler, 1)[0]
    assert first==predict_window(a, series[134:140], scaler)['prediction']
    for bad in [[1.], [float('nan')]*6, [float('inf')]*6]:
        try:
            predict_window(a, bad, scaler)
        except ValueError:
            pass
        else:
            raise AssertionError('Недопустимый ввод принят')
    step = manual_step()
    assert np.allclose(step['a'], [.18, .205])
    assert (step['prediction_after']-.6)**2 < step['loss']
    print(f'Проверки пройдены; ошибка градиента {max_error:.3e}')

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--lr', type=float, default=.05)
    parser.add_argument('--ee', type=float, default=.0001)
    parser.add_argument('--max-epochs', type=int, default=20000)
    parser.add_argument('--seeds', type=int, nargs='+', default=SEEDS)
    parser.add_argument('--out', type=Path, default=Path(__file__).resolve().parent/'lab4_results')
    parser.add_argument('--no-show', action='store_true')
    parser.add_argument('--no-input', action='store_true')
    parser.add_argument('--self-test', action='store_true')
    args = parser.parse_args()
    if args.self_test:
        self_test(); return
    if (not np.isfinite([args.lr, args.ee]).all() or args.lr<=0 or args.ee<=0
            or args.max_epochs<1 or len(args.seeds)<5 or min(args.seeds)<0
            or len(set(args.seeds))!=len(args.seeds)):
        parser.error('lr и ee > 0; max-epochs >= 1; >=5 различных seed >=0.')
    args.out.mkdir(parents=True, exist_ok=True)
    series = generate_series(); scaler = fit_scaler(series[:SPLIT])
    configs = [(p,N_BASE) for p in [2,4,6,12]]
    configs += [(P_BASE,n) for n in [1,2,8]]  # 6–4–1 уже включена.
    runs = []
    for p,n in configs:
        for seed in args.seeds:
            row = run_experiment(series, scaler, p, n, seed, args)
            runs.append(row)
            print(f'p={p:2} N={n} seed={seed}: эпох={row["epochs"]:5}, '
                  f'Es={row["Es"]:.7f}, test MAE={row["test"]["MAE"]:.6f}, '
                  f'порог={row["converged"]}', flush=True)
    groups = {(p,n): [r for r in runs if r['p']==p and r['n']==n] for p,n in configs}
    summaries = {key: summary(rows) for key,rows in groups.items()}
    best_key = min(configs, key=lambda key: summaries[key]['test_mae_mean'])
    # Конфигурация выбирается по среднему пяти запусков, seed — заранее первый.
    best = groups[best_key][0]; base = groups[(P_BASE,N_BASE)][0]
    prediction = recursive_forecast(restore(best), series[140-best['p']:140], scaler)
```

```
    error = np.abs(prediction-series[140:170])
    threshold = .05*float(np.ptp(series))
    exceed = np.flatnonzero(error>threshold)
    noise_runs = []
    for seed in args.seeds:
        row = run_experiment(series, scaler, 6, 4, seed, args, noise=.01)
        noise_runs.append(row)
        print(f'Шум 0.01, seed={seed}: эпох={row["epochs"]}, '
              f'test MAE={row["test"]["MAE"]:.6f}', flush=True)
    data = {'parameters': {'lr': args.lr, 'ee': args.ee, 'max_epochs': args.max_epochs,
                           'seeds': args.seeds}, 'numpy_version': np.__version__,
            'series': series.tolist(), 'scaler': scaler, 'runs': runs, 'base': base,
            'window_summary': [summaries[(p,4)] for p in [2,4,6,12]],
            'hidden_summary': [summaries[(6,n)] for n in [1,2,4,8]],
            'noise_runs': noise_runs, 'noise_summary': summary(noise_runs),
            'recursive': {'p': best['p'], 'n': best['n'], 'seed': best['seed'],
                          'prediction': prediction.tolist(), 'absolute_error': error.tolist(),
                          'one_step_error': np.abs(np.array(best['test_prediction'][:30])
                                                   -series[140:170]).tolist(),
                          'threshold': threshold,
                          'first_exceed_step': int(exceed[0]+1) if len(exceed) else None,
                          'metrics': metrics(series[140:170], prediction),
                          'one_step_metrics': metrics(series[140:170], best['test_prediction'][:30])},
            'demonstrations': demonstrations(restore(base),series,scaler),
            'manual': manual_step(args.lr)}
    (args.out/'results.json').write_text(json.dumps(data, ensure_ascii=False,
                              indent=2, allow_nan=False), encoding='utf-8')
    for name in ['window_summary','hidden_summary']:
        with (args.out/(name+'.csv')).open('w',encoding='utf-8-sig',newline='') as f:
            writer=csv.DictWriter(f,fieldnames=list(data[name][0]))
            writer.writeheader(); writer.writerows(data[name])
    print('\nБазовый эксперимент:', {k:base[k] for k in ['epochs','Es','train','test']})
    print('Лучшая конфигурация по среднему test MAE:',best_key)
    print('Первое превышение 5% размаха, шаг:',data['recursive']['first_exceed_step'])
    for row in data['demonstrations']:
        print('\n',row['label'],'входы:',np.round(row['window'],6))
        print('Нормированные:',np.round(row['normalized'],6))
        print(f'Прогноз норм.={row["prediction_normalized"]:.4f}; '
              f'исходный={row["prediction"]:.8f}; эталон={row["truth"]}; '
              f'ошибка={row["absolute_error"]}')
    visualize(data,args.out,not args.no_show)
    print('Результаты сохранены:',args.out.resolve())
    if not args.no_input:
        interactive(restore(base),series,scaler)

if __name__=='__main__':
    main()
```